Intitulé du cours: Bases de Données

Chapitre 3 : les Requêtes SQL

Enseignante: Abid Arij

Année de formation 2025-2026

Objectifs du module

À la fin du module, l'apprenant doit être capable de :

- **Comprendre les bases des bases de données relationnelles**
- Comprendre ce qu'est une **base de données relationnelle**.
- Identifier les **tables**, **colonnes**, **lignes (enregistrements)** et **relations** entre tables.
- Connaître les concepts de **clé primaire** et **clé étrangère** pour garantir l'intégrité des données.
- **Maîtriser le langage SQL pour interroger une base de données**
- **Utiliser les fonctions d'agrégation**

Introduction à SQL

SQL (Structured Query Language) est le **langage standard** pour interagir avec une base de données relationnelle.

Il permet de :

- **Créer** des tables et bases de données
- **Insérer**, **modifier**, **supprimer** des données
- **Rechercher** des informations précises
- **Contrôler** les accès utilisateurs

Les SGBD qui utilisent SQL : **MySQL**, **PostgreSQL**, **SQL Server**, **Oracle**, **SQLite**, etc.

Le langage SQL

- SQL : Structured Query Language
- Permet d’accéder aux données de la base de données,
- C’est langage adapté aux bases de données relationnelles,
- Il existe sur tous les SGBD relationnels (Oracle, Access,...),
- Il est défini par une norme ISO,
- Il est utilisé dans les bases de données Oracles depuis 1979.

- SQL n'est pas un langage de programmation
- instruction SQL = requête
- résultat d'une requête = table

Caractéristiques

- SQL est un langage de définition de données :

  SQL est un langage de définition de données (**LDD**), c'est-à-dire qu'il permet de créer **des objets** (BD, tables, vues…) dans une base de données relationnelle, ainsi que d'en modifier ou en supprimer.

- SQL est un langage de manipulation de données :

  SQL est un langage de manipulation de données (**LMD**), cela signifie qu'il permet de sélectionner, insérer, modifier ou supprimer **des données** dans une table d'une base de données relationnelle.

- SQL est un langage de protections d'accès :

  Il est possible avec SQL de définir des permissions au niveau des utilisateurs d'une base de données. On parle de **DCL** (Data Control Language).

Structure d'une base de données relationnelle

- Une **base de données** contient plusieurs **tables**.
  Chaque **table** ressemble à un **tableur Excel**, avec :
- Des **colonnes** (ou **champs**) → définissent les types de données (nom, âge, prix…)
- Des **lignes** (ou **enregistrements**) → contiennent les données.

id	nom	prenom	age
1	Abid	Arij	25
2	Ben Ali	Sami	30

Les grandes familles de commandes SQL

Type de commande	Nom complet	Rôle principal	Exemples
DDL	Data Definition Language	Définir la structure de la base	CREATE, ALTER, DROP
DML	Data Manipulation Language	Manipuler les données	SELECT, INSERT, UPDATE, DELETE
DCL	Data Control Language	Gérer les permissions	GRANT, REVOKE
TCL	Transaction Control Language	Gérer les transactions	COMMIT, ROLLBACK, SAVEPOINT

Conseils Généraux

- SQL n'est pas sensible à la casse, cependant il est conseillé d'utiliser les mots réservés (commandes, le type de données ...) en majuscules
- Il ne faut pas oublier le point-virgule à la fin de chaque ligne de commande.
- Utiliser les deux traits -- pour mettre une ligne en commentaire
- Utiliser /* et */ pour mettre plusieurs lignes en commentaire
- Utiliser des noms significatifs pour les objets que vous créez
- Ne pas utiliser de mots réservés comme noms d'objets (tables, vue, colonne..)
- Mettre une clé primaire pour chacune des tables que vous créez
- Si vous avez à contrôler l'intégrité référentielle, alors il faudra déterminer l'ordre dans lequel vous allez créer vos tables.

les types de données

TYPE	DESCRIPTION
VARCHAR2 (taille_max)	Chaîne de caractères de longueur variable
CHAR (taille)	Chaîne de caractères de longueur fixe
NUMBER (taille)	nombre en virgule flottante avec jusqu'à 38 chiffres significatifs
NUMBER (nb chiffres, nb décimales)	nombre en virgule fixe
LONG (taille)	Données de type caractère pouvant stocker jusqu'à 2 gigabytes .
RAW(taille)	Données binaires
LONG RAW (taille)	Données binaires long devant être entrées en notation hexadécimale.
DATE	Date

Exemple

Catégorie	Type	Exemple d’utilisation
Nombres entiers	INT, SMALLINT, BIGINT	age INT
Nombres décimaux	DECIMAL, FLOAT, DOUBLE	prix DECIMAL(6,2)
Texte court	CHAR(n), VARCHAR(n)	nom VARCHAR(50)
Texte long	TEXT, MEDIUMTEXT	description TEXT
Date et heure	DATE, TIME, DATETIME, TIMESTAMP, YEAR	date_naissance DATE
Booléen	BOOLEAN	est_actif BOOLEAN
Valeurs prédéfinies	ENUM, SET	sexe ENUM('H','F')

SQL:
Langage de Définitions des Données
(LDD)

Commandes DDL (Définition des structures)

❖ **CREATE DATABASE**

Crée une nouvelle base :

CREATE DATABASE nom_base;

➡ CREATE DATABASE Gestion_Ecole;

❖ **USE**

Sélectionne une base de données :

USE nom_base;

➡ USE Gestion_Ecole;

Crée une table :

```
CREATE TABLE nom_table (
   nom_colonne  type_données
   [liste de contrainte de colonne]
   [DEFAULT valeur_par_défaut]
   [liste de contrainte de table]
);
```

```
CREATE TABLE Etudiants (
    id INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(50),
    prenom VARCHAR(50),
    age INT,
    ville VARCHAR(50)
);
```

- **Syntaxe générale:**

CREATE TABLE nom_table (
nom_colonne type_données
[liste de contrainte de colonne]
[DEFAULT valeur par défaut]
[liste de contrainte de table]
);

- **Les contraintes d'intégrité**

contrainte de colonne
[CONSTRAINT nom]
type de contrainte de colonne

type de contrainte de colonne

- PRIMARY KEY
- NOT NULL
- UNIQUE
- CHECK(condition sur valeur)
- REFERENCES nom de

contrainte de table
[CONSTRAINT nom]
type de contrainte de table

type de contrainte de table

- PRIMARY KEY (liste de nom de colonne)
- NOT NULL (liste de nom de colonne)
- UNIQUE (liste de nom de colonne)

❑ La création d'une table consiste à définir son nom, les colonnes qui la composent et leurs type

- **Syntaxe**

Pour créer une table, on fait recours à l'instruction suivante :

```
CREATE TABLE nom_table
( Nom_attribut 1   type1,
  Nom_attribut 2   type2,
   ……,
  PRIMARY KEY (nom_attribut_clé),
  FOREIGN KEY (nom_attribut_clé_étrangère)
  REFERENCES nom_table( nom_attribut_clé_étrangère ) );
```

Exemple de création de tables

Etudiant (NumEtud, NomEtud, PrenomEtud, #NumClass)
Classe (NumClass, NomClass)

```
CREATE TABLE CLASSE (
NUMCLASSE NUMBER(5),
NOMCLASSE VARCHAR2(10),
PRIMARY KEY(NUMCLASSE) ) ;

CREATE TABLE ETUDIANT (
NumEtud NUMBER(5) PRIMARY KEY,
NOM VARCHAR2(10),
PRENOM VARCHAR2(10),
NUMCLASSE VARCHAR2(10),
FOREIGN KEY(NUMCLASSE) REFERENCES CLASSE(NUMCLASSE) );
```

Exemple: créer la table client définie comme suit

CLIENT
NCLI: char (10)
NOM: char (32)
ADRESSE: char (60)
LOCALITE: char (30)
CAT[0-1]: char (2)
COMPTE: num (9,2)

```
Create table CLIENT (
NCLI char(10),
NOM char(32),
ADRESSE char(60),
LOCALITE char(30),
CAT char(2),
COMPTE decimal(9,2)  );
```

Commandes DML (Manipulation des Données)

INSERT INTO (Insérer des données)

❖ **Syntaxe :**

INSERT INTO nom_table (col1, col2, ...) VALUES (val1, val2, ...);

❖ **Exemple :**

INSERT INTO Etudiants (id_etudiant, nom, prenom, age, ville)
VALUES (1, 'Ben Ali', 'Amine', 21, 'Tunis');

❖ Tu peux insérer plusieurs lignes à la fois :

INSERT INTO Etudiants (nom, prenom, age, ville)
VALUES
('trabelsi, 'Ahmed', 23, 'Tunis'),
('Martin', 'Sophie', 20, 'Lyon');

UPDATE (Modifier des données)

```
UPDATE nom_table
SET col1 = nouvelle_valeur
WHERE condition;
```

```
UPDATE Etudiants
SET ville = 'Sfax'
WHERE nom = 'Ben Ali';
```

- Change la ville de l'étudiant nommé Ben Ali en "Sfax".

⚠ Attention : Sans la clause **WHERE,** toutes les lignes seront modifiées.

DELETE (Supprimer des données)

```
DELETE FROM nom_table WHERE condition;
```

```
DELETE FROM Etudiants WHERE age < 18;
```

- Supprime tous les étudiants de moins de 18 ans.

⚠ Sans WHERE, toutes les données seront supprimées de la table !

Requêtes SELECT

- **<u>SELECT simple:</u>**

SELECT colonne1, colonne2 FROM nom_table;

➡ SELECT nom, prenom FROM Etudiants;

- **<u>Toutes les colonnes :</u>**

SELECT * FROM Etudiants;

- **Généralités**

Il s'agit de chercher un ou plusieurs tuples de la base de données.

Syntaxe :

SELECT A1,...., A2

FROM R1,....., Rm

[WHERE [condition]

Group by <expression>;

Having <expression>;

Order by [DESC/ASC];]

- Ai un nom d'attribut (de colonne)
- Rj un nom de relation (de table)
- FROM définit à partir de quelles tables le résultat est calculé
- WHERE définit les prédicats de sélection du résultat
- Group by regroupe les lignes pour les valeurs égales dans les colonnes mentionnées
- Having sélectionne les groupes de lignes satisfaisant la condition
- Order by précise si le tri se fait de manière croissante (par défaut) ou décroissante

- Affiche toutes les colonnes et toutes les lignes de la table Etudiants.

```
SELECT * FROM Etudiants;
```

- Affiche uniquement les colonnes nom et ville.

```
SELECT nom, ville FROM Etudiants;
```

- **Filtrer les résultats (WHERE)**

⚙️ **Syntaxe :**

```
SELECT * FROM nom_table WHERE condition;
```

```
SELECT * FROM Etudiants WHERE ville = 'Tunis';
```

Affiche uniquement les étudiants qui habitent à Tunis.

Autres opérateurs possibles :

Opérateur	Signification	Exemple
=	Égal	age = 20
<> ou !=	Différent	ville <> 'Paris'
>, <, >=, <=	Comparaison numérique	age > 18
BETWEEN	Intervalle	age BETWEEN 18 AND 25
LIKE	Recherche avec motif	nom LIKE 'A%' (commence par A)
IN	Liste de valeurs	ville IN ('Tunis', 'Sousse')

Requêtes SELECT

WHERE (Filtrer les résultats):

SELECT * FROM nom_table WHERE condition;

```
SELECT * FROM Etudiants WHERE ville = 'Paris';
SELECT * FROM Etudiants WHERE age > 20;
SELECT * FROM Etudiants WHERE age BETWEEN 18 AND 25;
SELECT * FROM Etudiants WHERE ville IN ('Paris', 'Lyon');
```

Exemple:

Liste des étudiants qui ne s'appellent pas Ali

```
SELECT * FROM ETUDIANT
WHERE PRENOM != 'ALI';
```

- **Exemple** 2

Liste des étudiants ayant les NumEtd inférieurs à 100

```
SELECT * FROM ETUDIANT
WHERE NumEtd < 100;
```

- **Exemple** 3

Liste des étudiants ayant les NumEtd compris entre 100 et 200

```
SELECT * FROM ETUDIANT
WHERE NumEtd BETWEEN 100 AND 200 ;
```

ORDER BY (Trier les résultats)

Trier les résultats (ORDER BY)

▶ Syntaxe :

SELECT * FROM nom_table ORDER BY colonne [ASC|DESC];

- Trie les résultats selon la colonne **age** (du plus petit au plus grand).
  SELECT * FROM Etudiants ORDER BY age ASC;

- Trie du plus grand au plus petit.
  SELECT * FROM Etudiants ORDER BY age DESC;

Grouper les résultats (GROUP BY)

```
SELECT ville, COUNT(*) AS nombre_etudiants
FROM Etudiants
GROUP BY ville;
```

Donne le **nombre d'étudiants par ville**.

Combiner plusieurs conditions

```
SELECT * FROM Etudiants
WHERE ville = 'Tunis' AND age > 20;
```

▶ Étudiants de Tunis âgés de plus de 20 ans.

```
SELECT * FROM Etudiants
WHERE ville = 'Sousse' OR ville = 'Monastir';
```

▶ Étudiants vivant à Sousse ou Monastir.

On peut combiner :
- `AND` → les deux conditions doivent être vraies.
- `OR` → au moins une condition doit être vraie.
- `NOT` → inverse la condition.

Limiter le nombre de résultats (LIMIT)

❖ **Afficher les 3 premiers cours**

```
SELECT *
FROM Cours
LIMIT 3;
```

id_cours	nom_cours	coefficient
101	Mathématiques	4
102	Physique	3
103	Informatique	5

▶Explication :

LIMIT 3 → ne retourne que les 3 premières lignes du résultat.

❖ Afficher les 2 cours avec le coefficient le plus élevé

```
SELECT *
FROM Cours
ORDER BY coefficient DESC
LIMIT 2;
```

id_cours	nom_cours	coefficient
103	Informatique	5
101	Mathématiques	4

`ORDER BY coefficient DESC` → trie les lignes par coefficient décroissant.

DISTINCT (Éliminer les doublons)

SELECT DISTINCT colonne FROM nom_table;

id_cours	nom_cours	coefficient
101	Mathématiques	4
102	Physique	3
103	Informatique	5
104	Chimie	2
105	Biologie	3
106	Histoire	3

❖ **Afficher tous les coefficients uniques**

SELECT DISTINCT coefficient
FROM Cours;

coefficient
2
3
4
5

`DISTINCT` supprime les doublons. Ici, le coefficient `3` n'apparaît qu'une seule fois.

LIKE (Recherche par motif)

```
SELECT * FROM nom_table WHERE colonne LIKE 'motif';
```

```
SELECT * FROM Etudiants WHERE nom LIKE 'D%';   -- commence par D
SELECT * FROM Etudiants WHERE nom LIKE '%e';   -- finit par e
SELECT * FROM Etudiants WHERE nom LIKE '%an%';  -- contient "an"
```

Le rôle de GROUP BY

Objectif :

- GROUP BY sert à **regrouper les lignes** d'une table **ayant une valeur identique** dans une ou plusieurs colonnes.

Ensuite, on peut **appliquer des fonctions d'agrégation** comme :

- COUNT() → compter le nombre d'éléments
- SUM() → faire une somme
- AVG() → faire une moyenne
- MIN() → trouver la plus petite valeur
- MAX() → trouver la plus grande valeur

❖ Requête sans GROUP BY :

```
SELECT ville, COUNT(*)
FROM Etudiants;
```

❌ Erreur : MySQL te dira que tu dois utiliser **GROUP BY**
Pourquoi ? Parce que COUNT() veut compter **par ville**, mais tu n'as pas dit **comment regrouper** les villes.

❖ Requête correcte avec GROUP BY :

```
SELECT ville, COUNT(*) AS nombre_etudiants
FROM Etudiants
GROUP BY ville;
```

ville	nombre_etudiants
Tunis	3
Sousse	2
Sfax	1

Le SGBD (MySQL, etc.) a regroupé **toutes les lignes de même ville**

- Puis, il a **compté** combien il y en a dans chaque groupe.

Les fonctions d'agrégation

Fonction	Description	Exemple
COUNT()	Compte le nombre de lignes	SELECT COUNT(*) FROM etudiants;
AVG()	Moyenne	SELECT AVG(age) FROM etudiants;
MAX()	Valeur maximale	SELECT MAX(age) FROM etudiants;
MIN()	Valeur minimale	SELECT MIN(age) FROM etudiants;
SUM()	Somme	SELECT SUM(age) FROM etudiants;

Fonctions d'aggrégation

- Exemple:

STUDENTS [Number, Name, Address, Age]
INSCRIPTION [#StudNb, #CourseCode, Year, Time]
COURSE [CourseCode, CourseT itle, CreditNb, HoursNb]

❖ Trouver la moyenne d'age des étudiants habitant Blois

```
SELECT AVG (Age) FROM STUDENTS
WHERE Address = 'Blois' ;
```

❖ Combien d'étudiants avons nous?

```
SELECT count(*)
FROM STUDENTS  ;
```

Le rôle de HAVING

Objectif :

HAVING sert à **filtrer les groupes créés par GROUP BY**.

🆚 Différence avec WHERE :

WHERE filtre **les lignes avant le regroupement**

HAVING filtre **les groupes après le regroupement**

```
SELECT ville, COUNT(*) AS nb, AVG(age) AS moyenne
FROM Etudiants
WHERE age >= 20
GROUP BY ville
HAVING nb >= 2
ORDER BY moyenne DESC;
```

- WHERE age >= 20 → garde seulement les étudiants de 20 ans et plus.
- GROUP BY ville → groupe par ville.
- COUNT(*) AS nb → compte le nombre d'étudiants par ville.
- HAVING nb >= 2 → garde les villes ayant au moins 2 étudiants.
- ORDER BY moyenne DESC → trie les villes selon la moyenne d'âge décroissante.

Modification dans une table

- **Modification de la structure d'une table** :
  - Il y a trois types d'actions concernant la modification de structure d'une table :
    - Ajouter une ou plusieurs colonnes.
    - Supprimer une ou plusieurs colonnes.
    - Modifier les propriétés d'une ou de plusieurs colonnes.
  - Les commandes SQL relatives à ces actions :
    - **ADD**
    - **DROP**
    - **MODIFY**

Modification dans une table

- L'instruction **ALTER TABLE** permet d'ajouter ou de supprimer un **seul** champ à une table.
- Elle permet aussi la création et la suppression des liens entre les tables d'une base de données.

Suppression / Modification d'une table

❑ Modification des tables

Il existe plusieurs modifications que l'on peut effectuer sur une table donnée:

- **Ajout d'attributs**

Après avoir créé la base de données, des tâches de maintenance semblent être parfois nécessaires. D'où l'ajout d'un nouvel attribut:

ALTER TABLE nom_table
ADD (attribut type);

- **Exemple**

Ajouter le champ Numéro téléphone à la table Etudiant revient à écrire :

ALTER TABLE Etudiant
ADD (Num_tel varchar(10));

Suppression / Modification d'une table

- **Modification des attributs**

Après avoir créé la base de données, on peut modifier le type d'un attribut en utilisant l'instruction suivante :

ALTER TABLE nom_table
MODIFY (attribut type);

- **Exemple**

Modifier le nombre de chiffres du champ **Num_tel** de la table Etudiant nécessite le recours à l'instruction :

ALTER TABLE Etudiant
MODIFY (Num_tel varchar(8));

Supprimer une table : <DROP TABLE>

La commande DROP permet de supprimer un objet de la base de données. (Table, indexe, synonyme..)

Exemple: DROP TABLE personne;

Renommer une table : <RENAME>

Permet de renommer une table ou un objet de la base de données

Syntaxe: RENAME <Ancien_nom> TO <Nouveau_nom>;

Exemple: RENAME Employes TO EmployesInfo;

Vidange d'une table : < TRUNCATE>

Permet de supprimer toutes les lignes de la table et de libèrer l'espace utilisé par la table

Syntaxe: TRUNCATE TABLE nom_table;

Merci pour

votre Attetion